CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 6

Outline

1. Introduction to React & Next.js
2. JSX
3. React Components

Introduction to React & Next.js

1. Introduction to React & Next.js
2. JSX
3. React Components

What is React?

- A JavaScript library, not a framework (unlike Angular, which is more opinionated)
- Open-source project created by Facebook
- Used to build user interfaces (UI) on the front-end
- React is the view layer of an MVC application (Model View Controller)

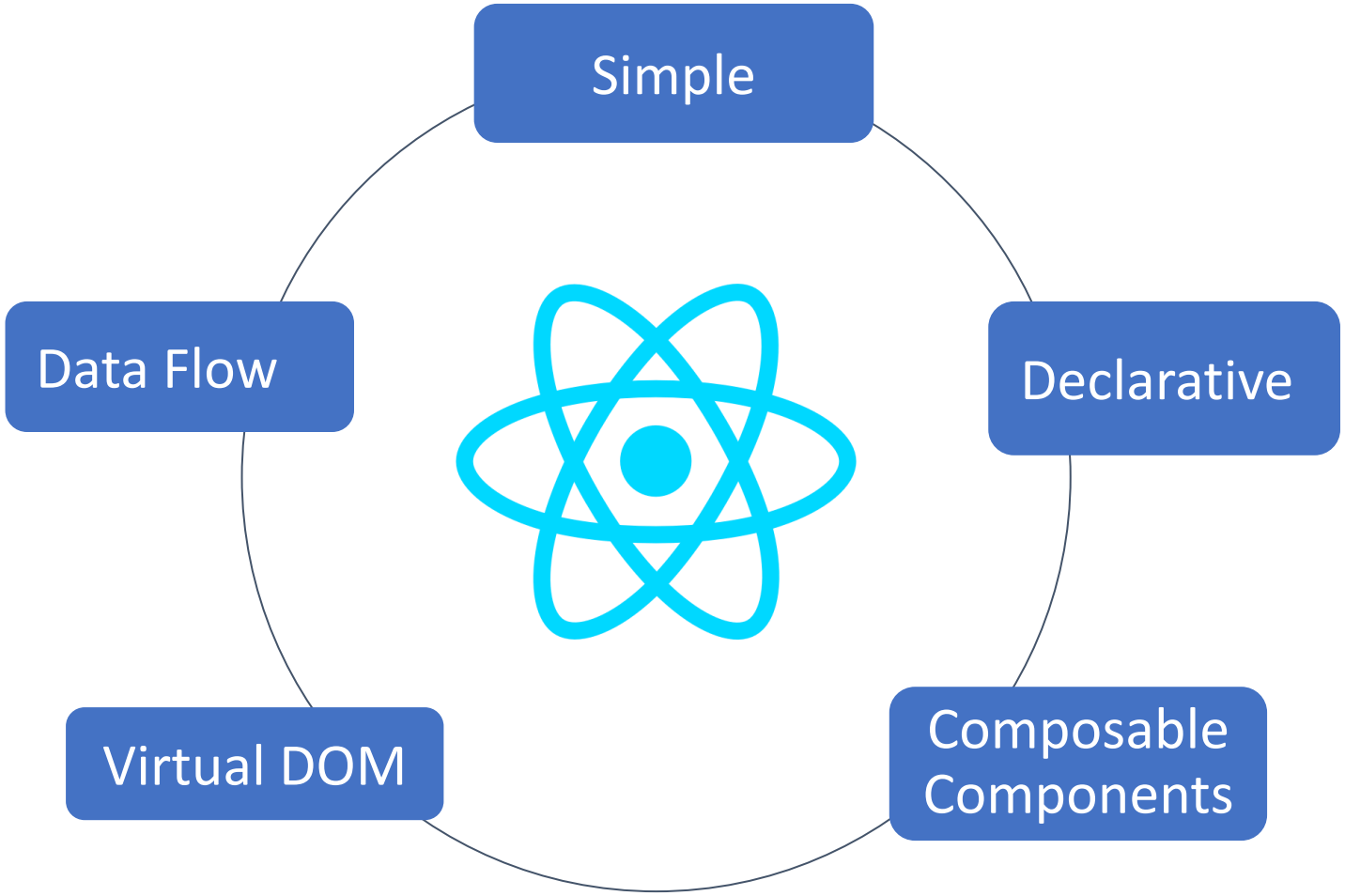
https://reactjs.org/

Why React?

Solves:

To prevent slow running of large applications with too many DOM operations in the background

Why React?


Simple
Declarative
Composable Components
Virtual DOM
Data Flow

Should Use React?

ÖZYEĞİN UNIVERSITY

Here's why you should use React:

- Works great for teams, strongly enforcing UI and workflow patterns
- UI code is readable and maintainable
- Componentized UI is the future of web development, and you need to start doing it now.

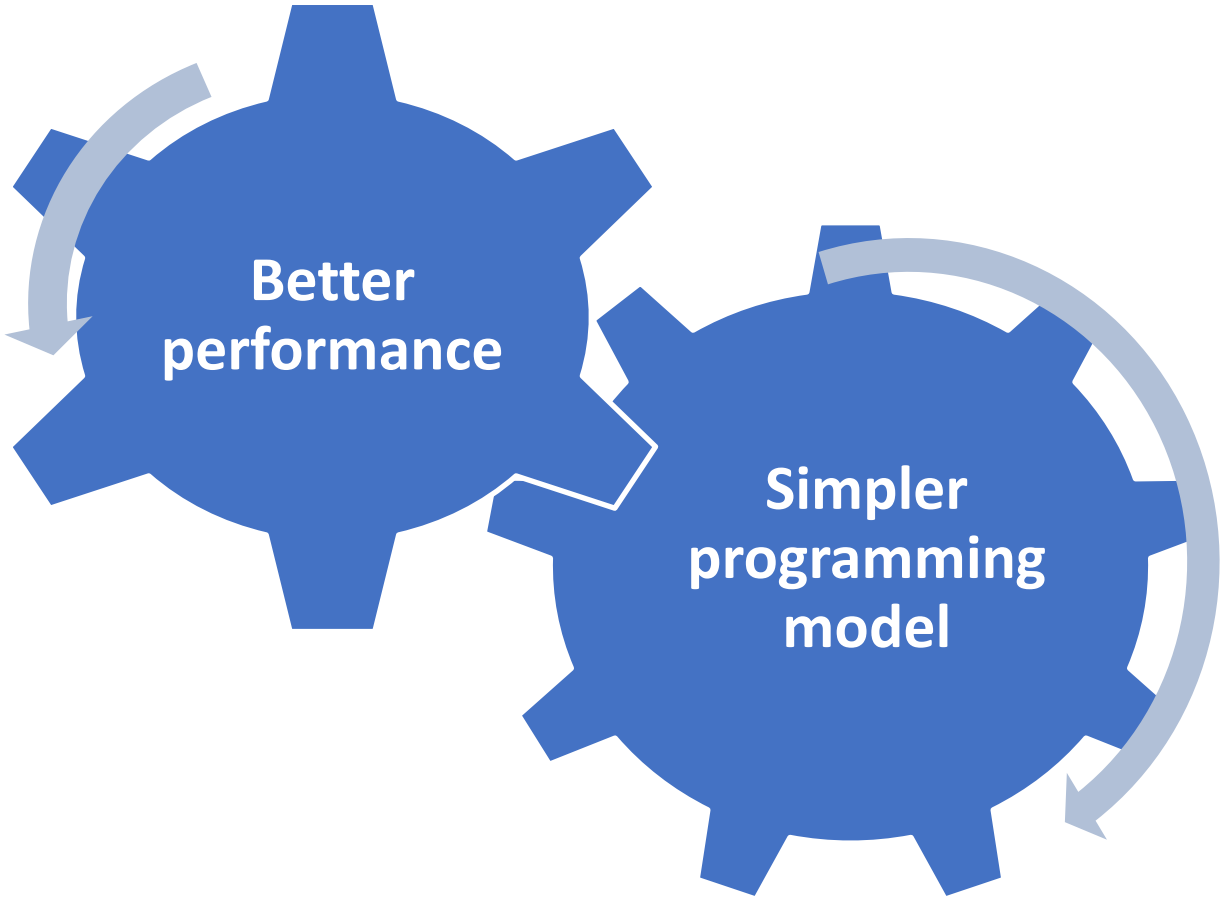
Should Use React?

ÖZYEĞİN UNIVERSITY

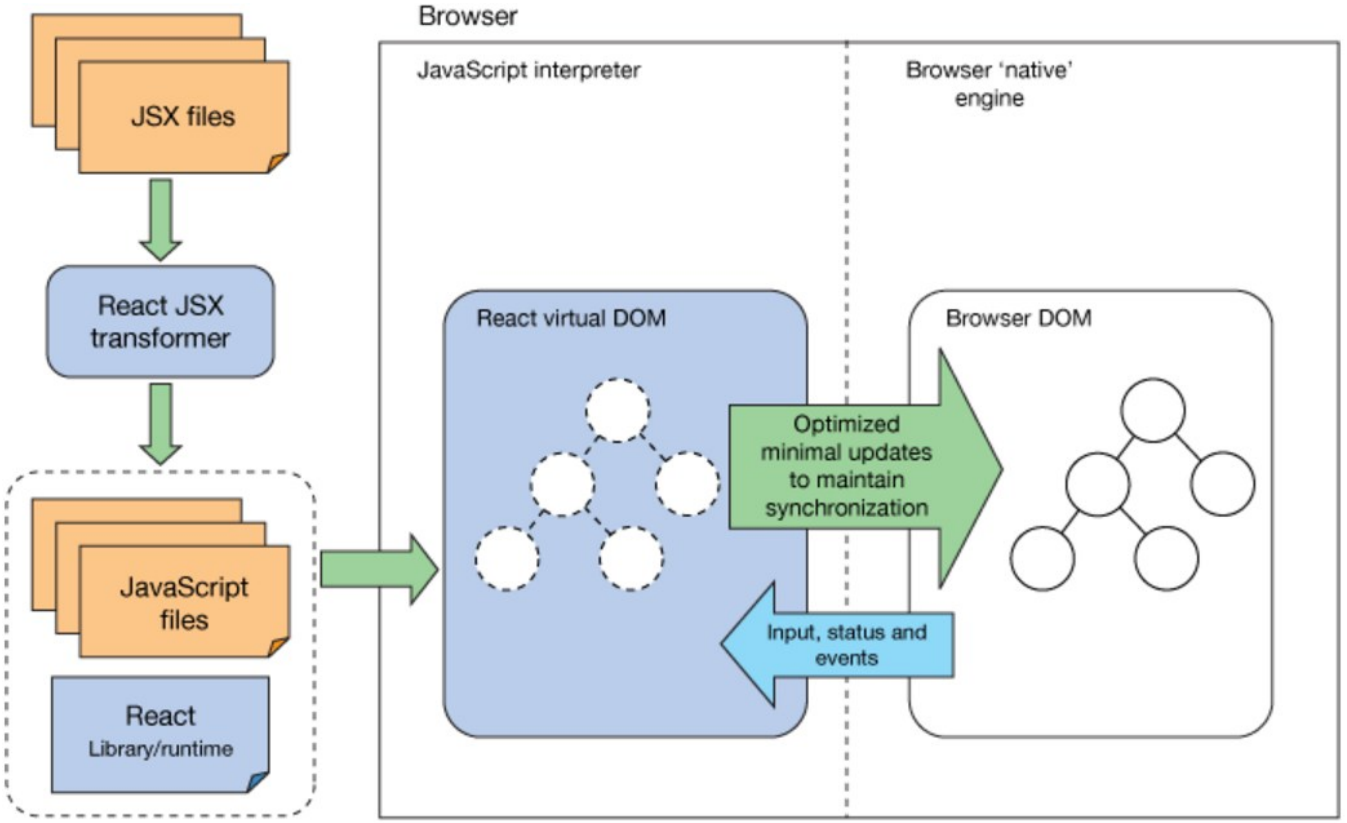
Think twice before you switch:

- React will slow you down at the start
- No support below IE8
- If your application doesn't have very much dynamic page updating

Virtual DOM



Virtual DOM is fast



Pure JavaScript Object
React computes the diff between this description and the in-memory DOM representation to compute the fastest way to update the browser

Separation of Concerns

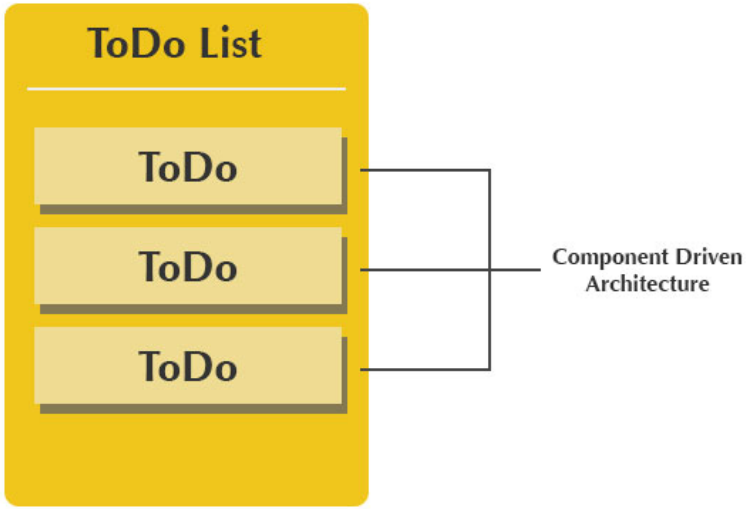
The ToDo List component deals with rendering each ToDo item

The ToDo Component deals with displaying the Todo Content

Component Oriented Development…

React components are very simple:

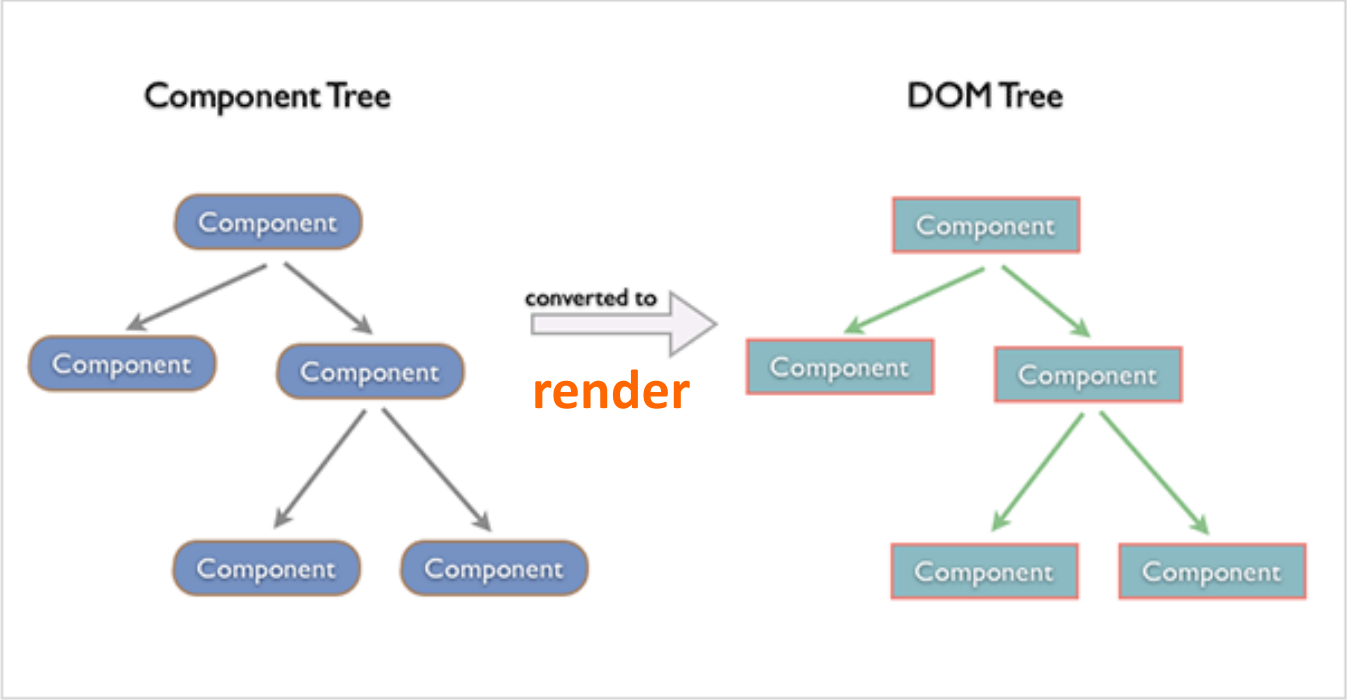
- Such as simple functions that take in props and state and render HTML can only render a single root node



...Component Oriented Development

1. FilterableProductTable (orange):
   - contains the entirety of the example
2. SearchBar (blue):
   - receives all user input
3. ProductTable (green):
   - displays and filters the data collection based on user input
4. ProductCategoryRow (turquoise):
   - displays a heading for each category
5. ProductRow (red):
   - displays a row for each product

Component Tree



render() function tells React how the DOM should look like at a given time. Does not produce a real DOM!

```
ReactDOM.render(<Simple message="Keep it Simple"/>, document.body);
```

Start a New React Project

- If you want to build a new app or a new website fully with React, we recommend picking one of the React-powered frameworks popular in the community.
- BUT: You can use React without a framework
  - However, most apps and sites eventually build solutions to common problems such as code-splitting, routing, data fetching, and generating HTML.
  - These problems are common to all UI libraries, not just React.

Production-grade React frameworks

- These frameworks support all the features you need to deploy and scale your app in production and are working towards supporting our full-stack architecture vision.
  - **Next.js :** React framework for building full-stack web applications. .
  - **Remix :** Full-stack React framework with nested routing.
  - **Gatsby :** Gatsby is a React framework for fast CMS-backed websites.
  - **Expo:** Expo is a React framework that lets you create universal Android, iOS, and web apps with truly native UIs

What is Next.js?

- Next.js is a React framework for building full-stack web applications.
- Built on ReactJS
- Let's you create React apps of any size—from a mostly static blog to a complex dynamic application.
- Also abstracts and automatically configures tooling needed for React, like bundling, compiling, and more. This allows you to focus on building your application instead of spending time with configuration

https://nextjs.org/

Some of the main Next.js features

- Routing
- Rendering
- Data Fetching
- Styling
- Optimizations
- TypeScript

Installation

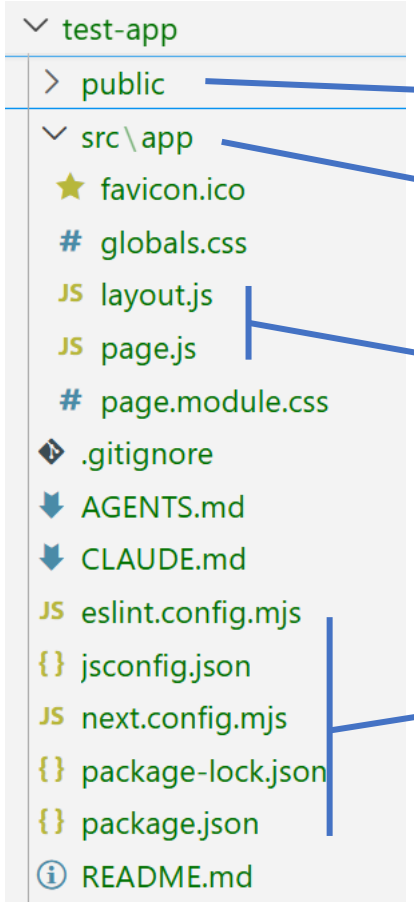
- System Requirements:
  - Node.js 18.17 or later.
  - macOS, Windows (including WSL), and Linux are supported.
- Automatic Installation(recommended)
  - use *create-next-app*, which sets up everything automatically for you
- Manual Installation
  - https://nextjs.org/docs/getting-started/installation#manual-installation

Create a project automatically

- To create a project, run:
  - npx create-next-app@latest
- On installation, you'll see the following prompts:

```
Need to install the following packages:
create-next-app@16.2.1
Ok to proceed? (y) y
√ What is your project named? ... test-app
√ Would you like to use the recommended Next.js defaults? » No, customize settings
√ Would you like to use TypeScript? ... No / Yes
√ Which linter would you like to use? » ESLint
√ Would you like to use React Compiler? ... No / Yes
√ Would you like to use Tailwind CSS? ... No / Yes
√ Would you like your code inside a `src/` directory? ... No / Yes
√ Would you like to use App Router? (recommended) ... No / Yes
√ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
√ Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? ... No / Yes
```

Project Structure



Stores static assets such as images, fonts, etc.
Files can be referenced by your code starting from the base URL (/)

Contains all the routes, components, and logic for your application, this is where you'll be mostly working from

These will be rendered when the user visits the root of your application (/).

Config Files

https://nextjs.org/docs/getting-started/project-structure

Run the Development Server

- Run `npm run dev` to start the development server.
  - Don't forget to change directory first!
- Visit `http://localhost:3000` to view your application.
- Edit `app/page.js` file and save it to see the updated result in your browser

Output of the new NextJS Project

NEXT.JS

To get started, edit the page.js file.

Looking for a starting point or more instructions?

Head over to **Templates** or the **Learning** center.

Deploy Now | Documentation

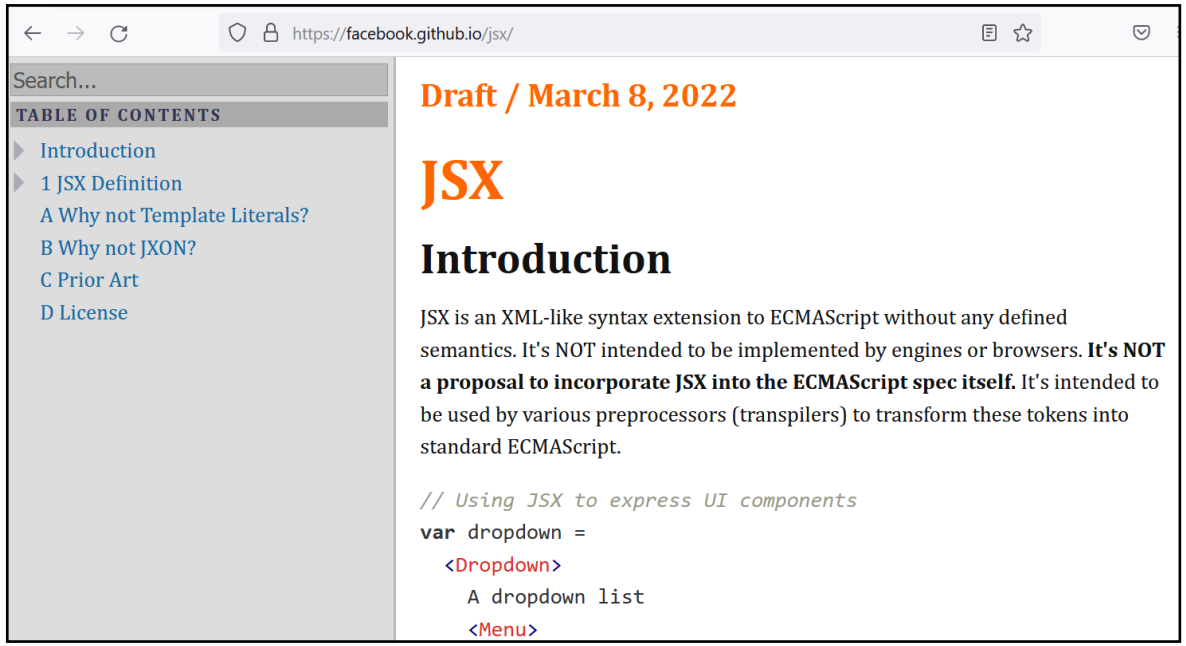
JSX

1. Introduction to ReactJS
2. JSX
3. React Components

JSX: JavaScript + XML

- JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside a JavaScript file.
- Let's you create JavaScript objects using HTML syntax
- JSX is not required to use React!
  - JSX is a syntax extension, while React is a JavaScript library

https://facebook.github.io/jsx/



JSX: JavaScript + XML

- The Web has been built on HTML, CSS, and JavaScript. For many years, web developers kept content in HTML, design in CSS, and logic in JavaScript—often in separate files! Content was marked up inside HTML while the page's logic lived separately in JavaScript:

```
<div>
  <p></p>
  <form>
  </form>
</div>
```

HTML

```
isLoggedIn() {...}
onClick() {...}
onSubmit() {...}
```

JavaScript

JSX: JavaScript + XML

- But as the Web became more interactive, logic increasingly determined content. JavaScript was in charge of the HTML! This is why **in React, rendering logic and markup live together in the same place—components.**

```
Sidebar() {
  if (isLoggedIn()) {
    <p>Welcome</p>
  } else {
    <Form />
  }
}
```

`Sidebar.js` React component

```
Form() {
  onClick() {...}
  onSubmit() {...}
  <form onSubmit>
    <input onClick />
    <input onClick />
  </form>
}
```

`Form.js` React component

Example

Here's what JSX looks like assigned to a variable.

JSX

```
const heading=<h1 className="site-heading">Hello,React</h1>;
```

Non-JSX

```
const heading = React.createElement (
      'h1',
     {className: 'site-heading'},'Hello, React!'
);
```

JSX Rules…

- JSX is closer to JavaScript, not HTML, so there are a few key differences to note when writing it.
- Return a single root element
  - you can use a
  - or you can write <> and </> instead

```
export default function Article() {
  return (
    <div>
        <h2>  ... </h2>
        <p>   ... </p>
    </div>
  )
}
```

```
export default function Article() {
  return (
    <>
        <h2>  ... </h2>
        <p>   ... </p>
    </>
  )
}
```

JSX Rules…

- Close all the tags
  - JSX requires tags to be explicitly closed: self-closing tags like <img> must become <img />
- camelCase all most of the things!
  - JavaScript has limitations on variable names. For example, their names can't contain dashes or be reserved words like `class`.
  - Since `class` is a reserved word, in React you write `className`

HTML		JSX
class	>	className
onclick	>	onClick

For more info about JSX

- Read
- https://react.dev/learn/writing-markup-with-jsx

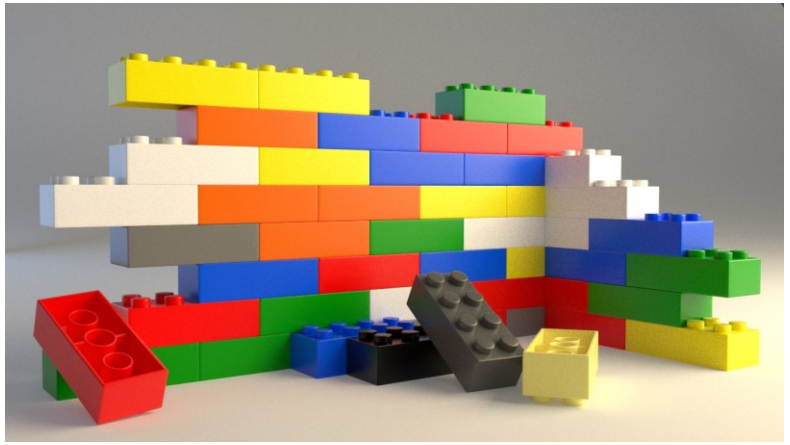
React Components

1. Introduction to ReactJS
2. JSX
3. React Components

Reusable Components

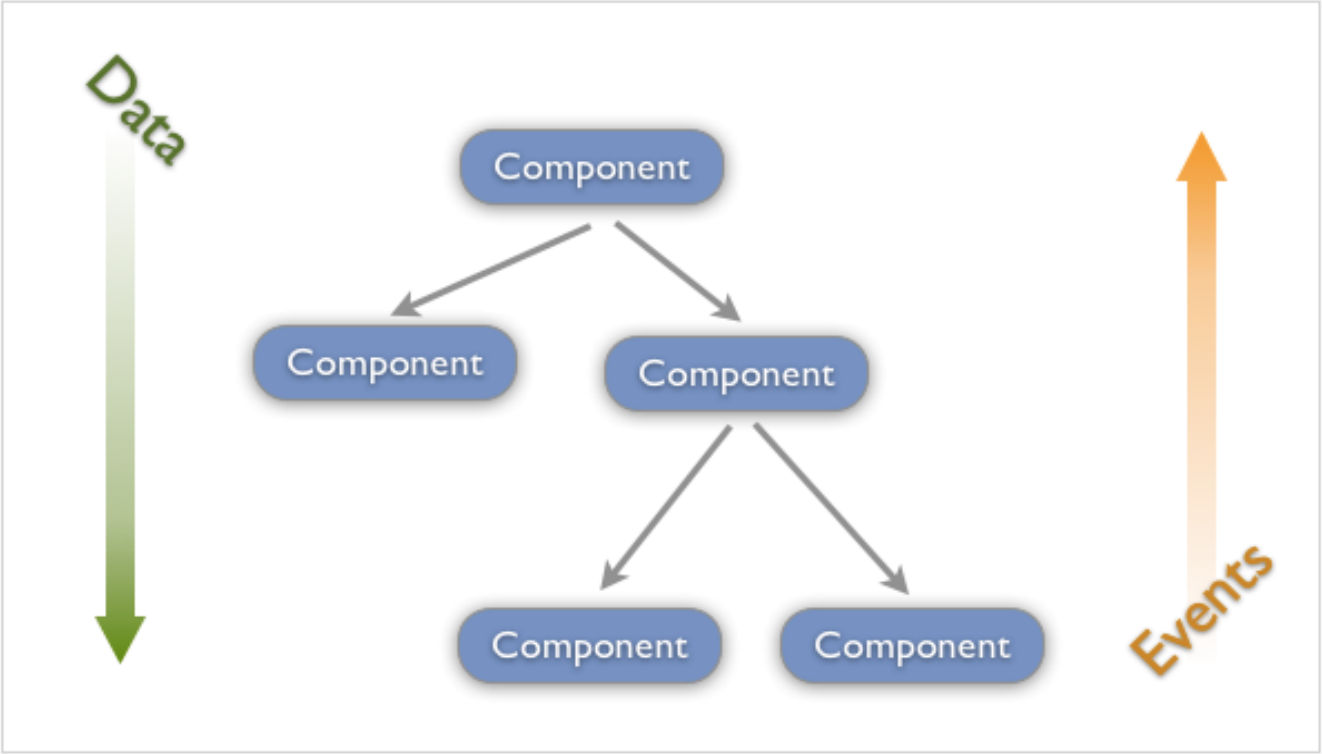
- Heart and soul of React

Faster Development Time → Fewer Bugs → Fewer bytes down the wire



Data Flow

- Data should always flow down



Components

- in React, rendering logic and markup live together in the same place—components.
- Each React component is a JavaScript function that may contain some markup that React renders into the browser.
- Almost everything in React consists of components
- Component types:
  - Function
  - Class

Functional Components

```
const Header = () => {
    return <div> My Web Application </div>
}
export default Header;
```

Class Components

- `Component` is the base class for the React components defined as JavaScript classes. **Class components are still supported by React but using them in new code are not recommended**.

```
import React, { Component } from 'react';
class SearchBar extends Component {
    render() {
        return (
            <div>
                Search: <input type="text" ></input>
                        <input type="button" value="Search"></input>
             </div>
         )
    }
}
export default SearchBar;
```

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr